



Assignment 1: Supervised learning competition

```
In [6]: # Install packages in notebook  
%pip install pyreadr optuna phik rpy2 --quiet
```

Note: you may need to restart the kernel to use updated packages.

```
In [ ]: import io  
  
import matplotlib.pyplot as plt  
import numpy as np  
import optuna  
import pandas as pd  
import phik  
import pyreadr  
import requests  
import xgboost as xgb  
from rpy2 import robjects  
from sklearn.ensemble import RandomForestRegressor  
from sklearn.feature_selection import RFE  
from sklearn.linear_model import ElasticNetCV  
from sklearn.metrics import root_mean_squared_error  
from sklearn.model_selection import (  
    KFold,  
    RandomizedSearchCV,  
    RepeatedKFold,  
    cross_val_score,  
)  
from sklearn.neighbors import KNeighborsRegressor  
from sklearn.pipeline import Pipeline, make_pipeline  
from sklearn.preprocessing import MinMaxScaler, RobustScaler, StandardScaler  
  
RANDOM_SEED = 14  
GOOGLE_COLAB = True # When using Google Colab set to True.  
DATA_SOURCE = "https://github.com/OssamaSijbesma/uu-data-analysis-and-machine-le  
  
optuna.logging.set_verbosity(optuna.logging.WARNING)  
np.random.seed(RANDOM_SEED)  
  
if GOOGLE_COLAB:  
    # Fetch files from online data source  
    train_response = requests.get(DATA_SOURCE + "raw/train.rds")  
    test_response = requests.get(DATA_SOURCE + "raw/test.rds")  
    train_rds = pyreadr.read_r(io.BytesIO(train_response.content))  
    test_rds = pyreadr.read_r(io.BytesIO(test_response.content))  
else:  
    # Fetch files from local data source  
    train_rds = pyreadr.read_r("./raw/train.rds")  
    test_rds = pyreadr.read_r("./raw/test.rds")  
  
# Create pandas data frames from files  
train_df = pd.DataFrame.from_dict(train_rds[None])  
test_df = pd.DataFrame.from_dict(test_rds[None])
```

Data description

We have the following information provided by the teachers on the dataset. On top of that, we did ask about the origin of the dataset as it could provide us with extra insights, sadly that information was not provided.

Attributes for both student-mat.csv (Math course) and student-por.csv (Portuguese language course) datasets:

1. school - student's school (binary: "GP" - Gabriel Pereira or "MS" - Mousinho da Silveira)
2. sex - student's sex (binary: "F" - female or "M" - male)
3. age - student's age (numeric: from 15 to 22)
4. address - student's home address type (binary: "U" - urban or "R" - rural)
5. famsize - family size (binary: "LE3" - less or equal to 3 or "GT3" - greater than 3)
6. Pstatus - parent's cohabitation status (binary: "T" - living together or "A" - apart)
7. Medu - mother's education (numeric: 0 - none, 1 - primary education (4th grade), 2 - 5th to 9th grade, 3 - secondary education or 4 - higher education)
8. Fedu - father's education (numeric: 0 - none, 1 - primary education (4th grade), 2 - 5th to 9th grade, 3 - secondary education or 4 - higher education)
9. Mjob - mother's job (nominal: "teacher", "health" care related, civil "services" (e.g. administrative or police), "at_home" or "other")
10. Fjob - father's job (nominal: "teacher", "health" care related, civil "services" (e.g. administrative or police), "at_home" or "other")
11. reason - reason to choose this school (nominal: close to "home", school "reputation", "course" preference or "other")
12. guardian - student's guardian (nominal: "mother", "father" or "other")
13. traveltime - home to school travel time (numeric: 1 - <15 min., 2 - 15 to 30 min., 3 - 30 min. to 1 hour, or 4 - >1 hour)
14. studytime - weekly study time (numeric: 1 - <2 hours, 2 - 2 to 5 hours, 3 - 5 to 10 hours, or 4 - >10 hours)
15. failures - number of past class failures (numeric: n if $1 \leq n < 3$, else 4)
16. schoolsup - extra educational support (binary: yes or no)
17. famsup - family educational support (binary: yes or no)
18. paid - extra paid classes within the course subject (Math or Portuguese) (binary: yes or no)
19. activities - extra-curricular activities (binary: yes or no)
20. nursery - attended nursery school (binary: yes or no)
21. higher - wants to take higher education (binary: yes or no)
22. internet - Internet access at home (binary: yes or no)
23. romantic - with a romantic relationship (binary: yes or no)
24. famrel - quality of family relationships (numeric: from 1 - very bad to 5 - excellent)
25. freetime - free time after school (numeric: from 1 - very low to 5 - very high)
26. goout - going out with friends (numeric: from 1 - very low to 5 - very high)
27. Dalc - workday alcohol consumption (numeric: from 1 - very low to 5 - very high)

28. Walc - weekend alcohol consumption (numeric: from 1 - very low to 5 - very high)

29. health - current health status (numeric: from 1 - very bad to 5 - very good)

30. absences - number of school absences (numeric: from 0 to 93)

and this is the outcome / prediction target

31. score - a normalized grade for an exam

```
In [8]: # Display the first few rows of the training dataset
train_df.head()
```

```
Out[8]:
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	int
0	GP	F	18.0	U	GT3	T	3.0	2.0	other	services	...	
1	GP	F	17.0	U	GT3	T	3.0	4.0	services	other	...	
2	GP	M	17.0	U	GT3	T	2.0	3.0	other	other	...	
3	GP	M	18.0	R	GT3	T	4.0	3.0	teacher	services	...	
4	GP	F	16.0	U	GT3	T	1.0	3.0	at_home	services	...	

5 rows × 31 columns



```
In [9]: # Display summary statistics for the training dataset
train_df.describe()
```

```
Out[9]:
```

	age	Medu	Fedu	traveltime	studytime	failures	fam
count	316.000000	316.000000	316.000000	316.000000	316.000000	316.000000	316.000000
mean	16.753165	2.737342	2.503165	1.452532	2.034810	0.370253	3.933500
std	1.302923	1.105828	1.102662	0.690728	0.840667	0.788167	0.904300
min	15.000000	0.000000	0.000000	1.000000	1.000000	0.000000	1.000000
25%	16.000000	2.000000	2.000000	1.000000	1.000000	0.000000	4.000000
50%	17.000000	3.000000	2.000000	1.000000	2.000000	0.000000	4.000000
75%	18.000000	4.000000	3.000000	2.000000	2.000000	0.000000	5.000000
max	22.000000	4.000000	4.000000	4.000000	4.000000	3.000000	5.000000



```
In [10]: # Display unique values for each categorical feature
for col in train_df.select_dtypes(include=["category"]).columns:
    print(f"{col}: {train_df[col].unique().tolist()}")
```

```

school: ['GP', 'MS']
sex: ['F', 'M']
address: ['U', 'R']
famsize: ['GT3', 'LE3']
Pstatus: ['T', 'A']
Mjob: ['other', 'services', 'teacher', 'at_home', 'health']
Fjob: ['services', 'other', 'at_home', 'teacher', 'health']
reason: ['other', 'course', 'home', 'reputation']
guardian: ['mother', 'father', 'other']
schoolsup: ['no', 'yes']
famsup: ['no', 'yes']
paid: ['no', 'yes']
activities: ['no', 'yes']
nursery: ['yes', 'no']
higher: ['yes', 'no']
internet: ['yes', 'no']
romantic: ['yes', 'no']

```

```

In [11]: target_col = "score"

# Phik ( $\phi_k$ ) is a correlation coefficient that works consistently between categor
# ordinal and interval variables, captures non-linear dependency and reverts to
# coefficient in case of a bivariate normal input distribution.
correlations = train_df.phik_matrix()[target_col]

# Create a table with data types, missing values, unique values, and phik correl
summary = pd.DataFrame(
    {
        "dtype": train_df.dtypes.astype(str),
        "missing": train_df.isna().sum(),
        "unique": train_df.nunique(),
        "phik_correlation": correlations.reindex(train_df.columns),
    }
).sort_values("phik_correlation", ascending=False)

# Display the table.
display(summary)

```

```

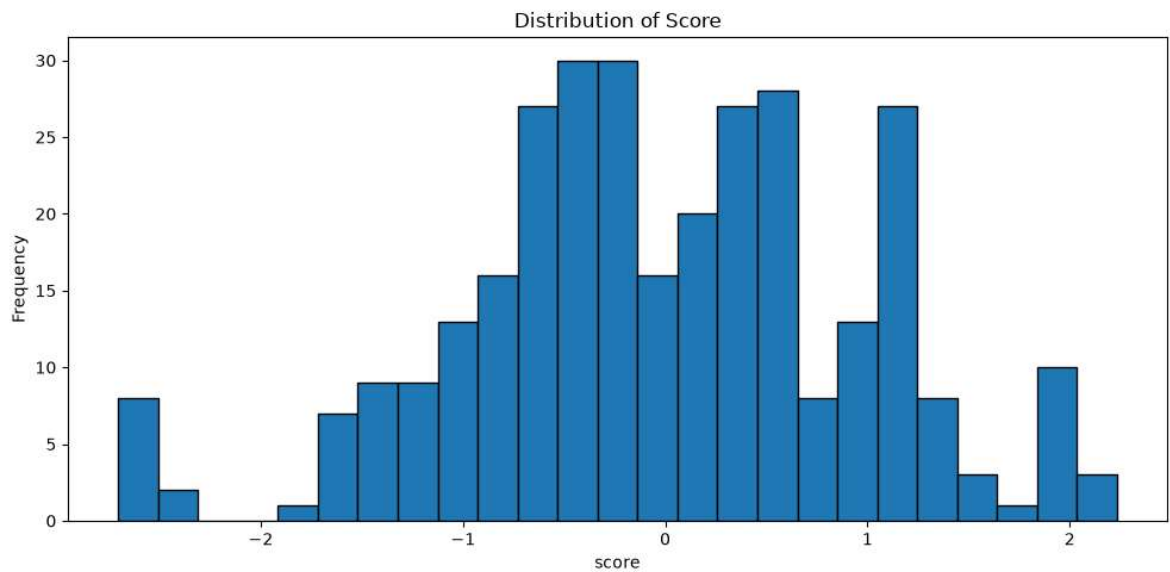
interval columns not set, guessing: ['age', 'Medu', 'Fedu', 'traveltime', 'studyt
ime', 'failures', 'famrel', 'freetime', 'goout', 'Dalc', 'Walc', 'health', 'absen
ces', 'score']

```


	dtype	missing	unique	phik_correlation
score	float64	0	169	1.000000
failures	float64	0	4	0.419492
schoolsup	category	0	2	0.312156
paid	category	0	2	0.276972
romantic	category	0	2	0.242853
goout	float64	0	5	0.191146
Medu	float64	0	5	0.183140
school	category	0	2	0.177013
Walc	float64	0	5	0.156285
reason	category	0	4	0.154200
address	category	0	2	0.148967
famrel	float64	0	5	0.122732
higher	category	0	2	0.109079
Pstatus	category	0	2	0.108985
studytime	float64	0	4	0.103432
sex	category	0	2	0.086737
nursery	category	0	2	0.086309
Mjob	category	0	5	0.083641
traveltime	float64	0	4	0.075807
Dalc	float64	0	5	0.008892
age	float64	0	8	0.000000
Fedu	float64	0	5	0.000000
famsize	category	0	2	0.000000
guardian	category	0	3	0.000000
Fjob	category	0	5	0.000000
famsup	category	0	2	0.000000
activities	category	0	2	0.000000
internet	category	0	2	0.000000
freetime	float64	0	5	0.000000
health	float64	0	5	0.000000
absences	float64	0	31	0.000000

```
In [12]: # Display the distribution of the outcome variable
plt.figure(figsize=(10, 5))
```

```
plt.hist(train_df["score"], bins=25, edgecolor="k")
plt.xlabel("score")
plt.ylabel("Frequency")
plt.title("Distribution of Score")
plt.tight_layout()
plt.show()
```



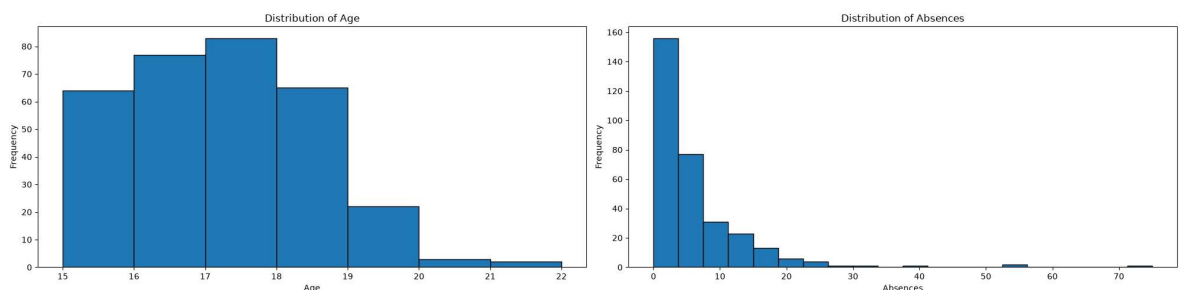
The scores seem to follow a normal distribution.

```
In [13]: # plot all distributions
plt.figure(figsize=(20, 5))

# left plot the distribution of age
plt.subplot(1, 2, 1)
plt.hist(train_df["age"], bins=7, edgecolor="k")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.title("Distribution of Age")

# Right plot the distribution of absences
plt.subplot(1, 2, 2)
plt.hist(train_df["absences"], bins=20, edgecolor="k")
plt.xlabel("Absences")
plt.ylabel("Frequency")
plt.title("Distribution of Absences")

plt.tight_layout()
plt.show()
```



Here we display the distributions of age and absences side by side.

There seems to be only a few observations on people who are 20+.

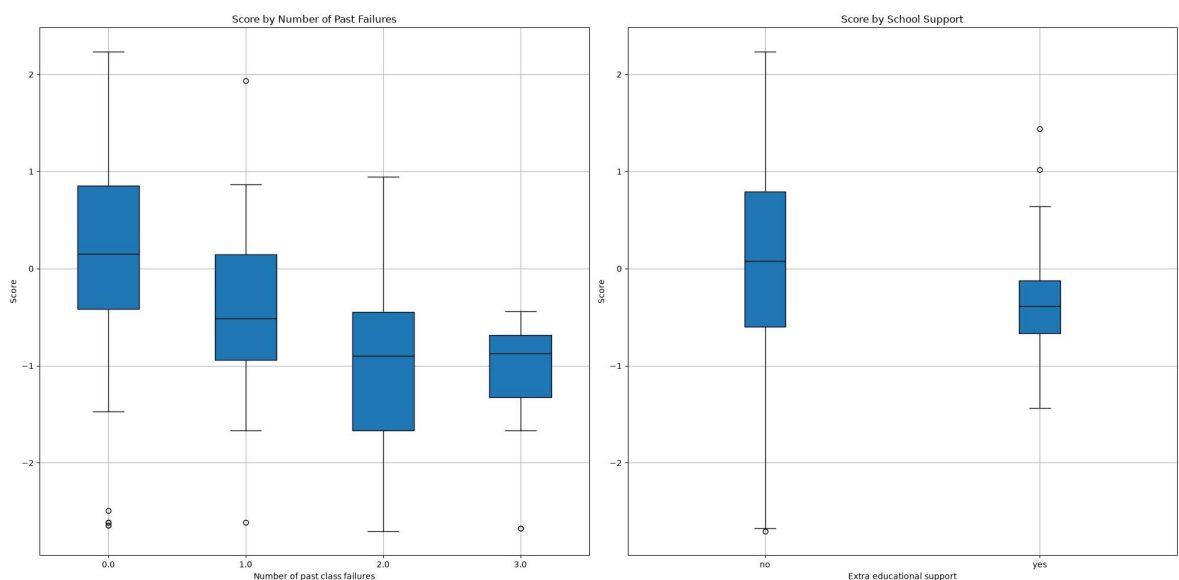
Absences seems heavily skewed to the left.

```
In [14]: # Create side-by-side box plots for the 2 variables most correlated with 'score'
fig, axes = plt.subplots(1, 2, figsize=(20, 10))

# Left plot: plot 'failures' against 'score'
train_df.boxplot(
    column="score",
    by="failures",
    ax=axes[0],
    patch_artist=True, # needed to fill boxes with color
    boxprops=dict(facecolor="C0"),
    medianprops=dict(color="black"),
    whiskerprops=dict(color="black"),
)
axes[0].set_xlabel("Number of past class failures")
axes[0].set_ylabel("Score")
axes[0].set_title("Score by Number of Past Failures")
fig.suptitle("")

# Right plot: plot 'schoolsup' against 'score'
train_df.boxplot(
    column="score",
    by="schoolsup",
    ax=axes[1],
    patch_artist=True,
    boxprops=dict(facecolor="C0"),
    medianprops=dict(color="black"),
    whiskerprops=dict(color="black"),
)
axes[1].set_xlabel("Extra educational support")
axes[1].set_ylabel("Score")
axes[1].set_title("Score by School Support")
fig.suptitle("")

plt.tight_layout()
plt.show()
```



More failures seem to show lower scores, as expected

Extra educational support seems to show a lower score

```

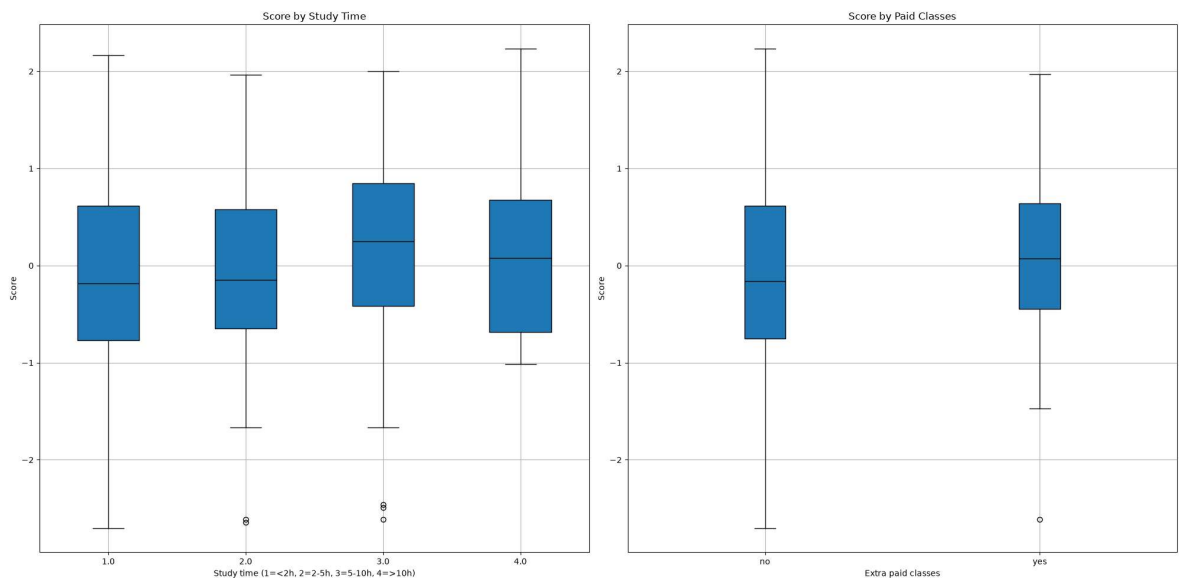
In [15]: # Plot score distributions by study time and paid classes
fig, axes = plt.subplots(1, 2, figsize=(20, 10))

train_df.boxplot(
    column="score",
    by="studytime",
    ax=axes[0],
    patch_artist=True,
    boxprops=dict(facecolor="C0"),
    medianprops=dict(color="black"),
    whiskerprops=dict(color="black"),
)
axes[0].set_xlabel("Study time (1=<2h, 2=2-5h, 3=5-10h, 4=>10h)")
axes[0].set_ylabel("Score")
axes[0].set_title("Score by Study Time")

train_df.boxplot(
    column="score",
    by="paid",
    ax=axes[1],
    patch_artist=True,
    boxprops=dict(facecolor="C0"),
    medianprops=dict(color="black"),
    whiskerprops=dict(color="black"),
)
axes[1].set_xlabel("Extra paid classes")
axes[1].set_ylabel("Score")
axes[1].set_title("Score by Paid Classes")

fig.suptitle("")
plt.tight_layout()
plt.show()

```



More study time indicates a low chance of low scores

Attending extra paid classes also seems to improve scores

The dataset consists of a mix of categorical and numerical features used to predict a standardized exam score. Our first impression is that the dataset has been heavily preprocessed, with most numerical variables standardized, except for age and absences.

The target variable combines scores from both the Portuguese and Mathematics courses. While the predictor variables do not appear inherently biased toward either subject, this assumption cannot be verified definitively since the dataset was provided in combined form. We have visualized the features that we expect has the largest influence on the score based on the correlation.

Model description

We are comparing 4 supervised learning models: K-nearest neighbours (KNN), random forest, extreme gradient boosting (XGBoost) and Elastic Net regression.

KNN predicts the score of a student by averaging the K most similar students across all predictor variables and measure similarity based on a distance metric. KNN is a simple method that does not make any assumptions on the distribution of the data, however, it may not be the most suitable in our case. Our data has 30 features (dimensions), which exponentially increases the volume of the feature space so the density of the data points is likely very sparse. Furthermore, many of our features are categorical and KNN requires them to be one-hot encoded, which further increases the number of dimensions.

Random forest is a bagging ensemble that fits many decision trees on bootstrap samples of the training data and averages their predictions. At each split only a number of predictors is considered, which decorrelates the trees and reduces variance. This technique generally has a high accuracy rate and can handle a mix of categorical and numeric data well, which is an advantage compared to KNN. Given that our dataset mixes numerical and categorical variables, random forest is likely to be a good fit and requires little preprocessing beyond one-hot encoding for categorical variables.

XGBoost is a boosting ensemble that builds trees sequentially, where each new tree is fit to the residual errors of the previous ensemble. Compared to random forest, XGBoost reduces both bias and variance by combining many weak learners into a strong one. Boosting is considered to be state of the art when considering tabular data like ours, however, hyperparameters need to be tuned carefully so that the model does not overfit.

Elastic Net is a linear regression model with two penalties added to the squared error - an L1 penalty that shrinks weak coefficients to exactly zero and an L2 penalty that shrinks them smoothly. It suits our data because with 39 columns for only 316 students, the penalties reduce overfitting and automatically drop irrelevant features. The L2 part also handles correlated predictors more stably than Lasso alone. The penalty strength (alpha) and the mix between the two penalties (l1_ratio) are tuned with cross-validation inside each training fold.

Data transformation and pre-processing

Here we do some basic transformations for testing purposes.

```
In [16]: # Target variable
y = train_df["score"]

# Raw features (unscaled)
X = train_df.drop(columns=["score"])
X_test = test_df.copy() # test.rds has no 'score' column

# Scaled features
X_scaled = X.copy()
X_scaled["age"] = MinMaxScaler().fit_transform(X_scaled[["age"]])
X_scaled["absences"] = RobustScaler().fit_transform(X_scaled[["absences"]])

# convert category or object values into dummies
X_dummy = pd.get_dummies(X, drop_first=True)
```

Prepare predictors for XGBoost Regression

I prepared the data to include a new column, as I thought two smart parents together produce smarter kids. I also created dummies for the categorical values with more than two values.

```
In [17]: # Create copy
X_XGB = X.copy()

# Create a new feature for the mean of parents' education
X_XGB["parent_edu_mean"] = (X_XGB["Medu"] + X_XGB["Fedu"]) / 2.0

# Convert nominal features to dummy variables
X_XGB = pd.get_dummies(
    X_XGB, columns=["Mjob", "Fjob", "reason", "guardian"], drop_first=True
)

# Scale values (not needed for XGB)
# X_XGB["age"] = MinMaxScaler().fit_transform(X_XGB[["age"]])
# X_XGB["absences"] = RobustScaler().fit_transform(X_XGB[["absences"]])
```

Prepare predictors for KNN

```
In [18]: # One-hot encode: turn categories into 0/1 columns so KNN doesn't assume a false
X_train_enc = pd.get_dummies(X, drop_first=True)
X_test_enc = pd.get_dummies(X_test, drop_first=True)

# Align test columns to match train exactly
X_test_enc = X_test_enc.reindex(columns=X_train_enc.columns, fill_value=0)

# Scale using statistics from TRAIN
scaler = StandardScaler()
X_train_knn = scaler.fit_transform(X_train_enc)
X_test_knn = scaler.transform(X_test_enc)
```

XGBoost

```
In [19]: # Baseline XGBoost model with default hyperparameters
xgb_base_model = xgb.XGBRegressor()
```

```

        tree_method="hist",
        enable_categorical=True,
        n_jobs=2,
        random_state=RANDOM_SEED,
    )

xgb_base_cv = KFold(n_splits=5, shuffle=True, random_state=RANDOM_SEED)
xgb_base_fold_mse = -cross_val_score(
    xgb_base_model,
    X_XGB,
    y,
    cv=xgb_base_cv,
    scoring="neg_mean_squared_error",
)
xgb_base_mse = xgb_base_fold_mse.mean()
xgb_base_rmse = np.sqrt(xgb_base_mse)
xgb_base_mse_std = xgb_base_fold_mse.std()

print(f"Baseline XGBoost CV MSE: {xgb_base_mse:.4f}")
print(f"Baseline XGBoost CV RMSE: {xgb_base_rmse:.4f}")
print(f"Baseline XGBoost CV MSE STD: {xgb_base_mse_std:.4f}")

```

Baseline XGBoost CV MSE: 0.9390
 Baseline XGBoost CV RMSE: 0.9690
 Baseline XGBoost CV MSE STD: 0.1947

```

In [20]: # Define the objective function for Optuna optimization (Bayesian Optimization w
def objective(trial):
    # Hyperparameter search space
    params = {
        "max_depth": trial.suggest_int("max_depth", 1, 10),
        "min_child_weight": trial.suggest_int("min_child_weight", 1, 20),
        "learning_rate": trial.suggest_float("learning_rate", 0.005, 0.2, log=True),
        "subsample": trial.suggest_float("subsample", 0.4, 1.0),
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.2, 1.0),
        "colsample_bylevel": trial.suggest_float("colsample_bylevel", 0.5, 1.0),
        "gamma": trial.suggest_float("gamma", 1e-4, 10.0, log=True),
        "reg_alpha": trial.suggest_float("reg_alpha", 1e-3, 50.0, log=True),
        "reg_lambda": trial.suggest_float("reg_lambda", 1e-2, 100.0, log=True),
        "max_bin": trial.suggest_int("max_bin", 64, 256),
        "n_estimators": trial.suggest_int("n_estimators", 100, 1000, step=50),
    }

    kf = KFold(n_splits=5, shuffle=True, random_state=RANDOM_SEED)
    scores = []

    for train_idx, val_idx in kf.split(X_XGB, y):
        # Split the data into training and validation sets for the current fold
        X_train, X_val = X_XGB.iloc[train_idx], X_XGB.iloc[val_idx]
        y_train, y_val = y.iloc[train_idx], y.iloc[val_idx]

        # Fit the model on the training set and validate on the validation set
        xgb_model = xgb.XGBRegressor(
            tree_method="hist",
            enable_categorical=True,
            n_jobs=2,
            eval_metric="rmse",
            random_state=RANDOM_SEED,
            **params,
        )

```

```

# Train the model on the training set and validate on the validation set
xgb_model.fit(X_train, y_train, eval_set=[(X_val, y_val)], verbose=False)

# Make predictions on the validation set
preds = xgb_model.predict(X_val)

# Calculate RMSE for the current fold
rmse = root_mean_squared_error(y_val, preds)

# Append the RMSE of the current fold to the scores list
scores.append(rmse)

return np.mean(scores)

# Terminate unpromising hyperparameter trials mid-evaluation
pruner = optuna.pruners.MedianPruner(
    n_startup_trials=5, # Wait 5 trials before pruning starts
    n_warmup_steps=10, # Let trees build for at least 10 rounds per trial
)

# Create Optuna study with median pruner
study = optuna.create_study(
    direction="minimize",
    sampler=optuna.samplers.TPESampler(seed=RANDOM_SEED),
    pruner=pruner,
)

# Optimize the objective function using Optuna (1000 trials)
study.optimize(objective, n_trials=1000, n_jobs=-1, show_progress_bar=True)

# Output Results
print(f"Optimized CV RMSE: {study.best_value:.4f}\n")

print("Hyperparameter selection:")
for key, value in study.best_params.items():
    print(f"{key} = {value}")

```

Best trial: 528. Best value: 0.858308: 100%|██████████| 1000/1000 [42:39<00:00, 2.56s/it]

Optimized CV RMSE: 0.8583

Hyperparameter selection:

```

max_depth = 2
min_child_weight = 3
learning_rate = 0.01177362136621535
subsample = 0.7621510832520896
colsample_bytree = 0.9562724167341276
colsample_bylevel = 0.9992013929065694
gamma = 0.31800453354524855
reg_alpha = 0.006237891210994992
reg_lambda = 0.045302102996026924
max_bin = 249
n_estimators = 400

```

In [21]: # Tuned XGBoost model with hyperparameters optimized using Optuna

```

xgb_tuned_model = xgb.XGBRegressor(
    tree_method="hist",
    enable_categorical=True,

```



```

n_jobs=2,
random_state=RANDOM_SEED,
**study.best_params,
)

# Perform cross-validation with the tuned XGBoost model
xgb_tuned_cv = KFold(n_splits=5, shuffle=True, random_state=RANDOM_SEED)
xgb_tuned_fold_mse = -cross_val_score(
    xgb_tuned_model,
    X_XGB,
    y,
    cv=xgb_tuned_cv,
    scoring="neg_mean_squared_error",
)

# Output the mean and standard deviation of the tuned XGBoost model's MSE across
xgb_tuned_mse = xgb_tuned_fold_mse.mean()
xgb_tuned_rmse = np.sqrt(xgb_tuned_mse)
xgb_tuned_mse_std = xgb_tuned_fold_mse.std()

# Print final results
print("XGBoost cross-validation results:")
print(f"Baseline MSE: {xgb_base_mse:.4f}")
print(f"Baseline RMSE: {xgb_base_rmse:.4f}")
print(f"Baseline MSE STD: {xgb_base_mse_std:.4f}")
print(f"Tuned MSE: {xgb_tuned_mse:.4f}")
print(f"Tuned RMSE: {xgb_tuned_rmse:.4f}")
print(f"Tuned MSE STD: {xgb_tuned_mse_std:.4f}")

```

XGBoost cross-validation results:
Baseline MSE: 0.9390
Baseline RMSE: 0.9690
Baseline MSE STD: 0.1947
Tuned MSE: 0.7398
Tuned RMSE: 0.8601
Tuned MSE STD: 0.0947

Random Forest Regression

```

In [22]: # Baseline RF model with default hyperparameters
rf_model = RandomForestRegressor(random_state=RANDOM_SEED)

rf_cv = cross_val_score(rf_model, X_dummy, y, cv=5, scoring="neg_mean_squared_er

rf_base_mse = -(rf_cv).mean()
rf_base_rmse = np.sqrt(rf_base_mse)
rf_base_std = rf_cv.std()

print(f"Best CV MSE: {rf_base_mse:.4f}")
print(f"Best CV RMSE: {rf_base_rmse:.4f}")
print(f"STD across folds: {rf_base_std:.4f}")

```

Best CV MSE: 0.7709
Best CV RMSE: 0.8780
STD across folds: 0.0720

```

In [23]: # RF model with feature selection and hyperparameter tuning

# Build pipeline where RFE selects important features, then the model is fit on

```

```

# The RF inside RFE only needs to give a reasonable importance ranking,
# so fewer trees and limited depth is used to decrease runtime
rf_pipeline = Pipeline(
    steps=[
        (
            "feature_selection",
            RFE(
                estimator=RandomForestRegressor(
                    n_estimators=50, max_depth=10, random_state=RANDOM_SEED
                ),
                step=3,
            ),
        ), # removing 3 features per round decreases run time
        ("model", RandomForestRegressor(random_state=42)),
    ]
)

# Provide a range of parameters for hyperparameter tuning
param_distributions = {
    # Choose randomly the number of features to select with RFE
    "feature_selection_n_features_to_select": [5, 10, 15, 20, 25],
    "model_n_estimators": [100, 200, 300, 500],
    "model_max_features": ["sqrt", "log2", 0.5, 1.0],
    "model_max_depth": [None, 5, 10, 15],
    "model_min_samples_split": [2, 5, 10],
    "model_min_samples_leaf": [1, 2, 4],
}

# Within one fold of the CV, RFE runs iteratively and picks out n features,
# and the RF is trained on those features.
# Running the RFE one fold at a time prevents data Leakage during CV.
rf_search = RandomizedSearchCV(
    estimator=rf_pipeline,
    param_distributions=param_distributions,
    n_iter=100, # Only 100 iterations is used since RFE increases run time
    cv=5,
    scoring="neg_mean_squared_error",
    n_jobs=-1,
    random_state=42,
    verbose=1,
)

rf_search.fit(X_dummy, y)

best = rf_search.best_index_
rf_tuned_mse = -rf_search.best_score_
rf_tuned_rmse = np.sqrt(-rf_search.best_score_)
rf_tuned_std = rf_search.cv_results_["std_test_score"][best]

print(f"Best Parameters: {rf_search.best_params_}")
print(f"Best CV MSE: {rf_tuned_mse:.4f}")
print(f"Best CV RMSE: {rf_tuned_rmse:.4f}")
print(f"STD across folds: {rf_tuned_std:.4f}")

# Selected features from the best pipeline (refit on all the data)
best_pipeline = rf_search.best_estimator_
rfe_selector = best_pipeline.named_steps["feature_selection"]
selected_feature_names = X_dummy.columns[rfe_selector.support_]

```

```
print(f"\nRFE selected {len(selected_feature_names)} features:")
print(list(selected_feature_names))
```

Fitting 5 folds for each of 100 candidates, totalling 500 fits

Best Parameters: {'model__n_estimators': 500, 'model__min_samples_split': 2, 'model__min_samples_leaf': 4, 'model__max_features': 0.5, 'model__max_depth': 5, 'feature_selection__n_features_to_select': 15}

Best CV MSE: 0.7480

Best CV RMSE: 0.8649

STD across folds: 0.0449

RFE selected 15 features:

```
['age', 'Medu', 'Fedu', 'studytime', 'failures', 'famrel', 'freetime', 'goout',
'Walc', 'health', 'absences', 'sex_M', 'Mjob_other', 'schoolsup_yes', 'activities_yes']
```

KNN

```
In [24]: k_values = range(1, 31)
cv_scores = []

kf = KFold(n_splits=5, shuffle=True, random_state=RANDOM_SEED)

for k in k_values:
    knn = KNeighborsRegressor(n_neighbors=k)
    scores = cross_val_score(
        knn, X_train_knn, y, cv=kf, scoring="neg_mean_squared_error"
    )
    cv_scores.append(-scores.mean())

best_k = k_values[np.argmin(cv_scores)]
print(f"Best k: {best_k}, CV MSE: {min(cv_scores):.3f}")
```

Best k: 16, CV MSE: 0.861

```
In [25]: # Baseline KNN using default k=5 (no tuning)
knn_baseline = KNeighborsRegressor()
baseline_scores = cross_val_score(
    knn_baseline, X_train_knn, y, cv=kf, scoring="neg_mean_squared_error"
)
knn_baseline = KNeighborsRegressor() # default n_neighbors=5
baseline_scores = cross_val_score(
    knn_baseline, X_train_knn, y, cv=kf, scoring="neg_mean_squared_error"
)

knn_base_mse = (-baseline_scores).mean()
knn_base_rmse = np.sqrt(knn_base_mse)
knn_base_std = (-baseline_scores).std()

# --- After hyperparameter tuning: best k from CV search ---
knn_tuned = KNeighborsRegressor(n_neighbors=best_k) # best_k = 28, found earlier
tuned_scores = cross_val_score(
    knn_tuned, X_train_knn, y, cv=kf, scoring="neg_mean_squared_error"
)

knn_tuned_mse = (-tuned_scores).mean()
knn_tuned_rmse = np.sqrt(knn_tuned_mse)
knn_tuned_std = (-tuned_scores).std()
```

```

print(
    f"first model\n MSE:{knn_base_mse:.4f}\n RMSE:{knn_base_rmse:.4f}\n STD:{
)
print(
    f"after hyperparameter tuning\n MSE:{knn_tuned_mse:.4f}\n RMSE:{knn_tuned_
)

```

```

first model
  MSE:1.0202
  RMSE:1.0100
  STD:0.0843
after hyperparameter tuning
  MSE:0.8609
  RMSE:0.9279
  STD:0.0880

```

Elastic net

```

In [26]: train, test = train_df, test_df
         target_col = "score"

```

```

In [27]: # Features: numeric columns plus one-hot encoded categoricals
y_en = train[target_col]
feat_cols = train.drop(columns=[target_col]).columns

X_en = pd.get_dummies(train[feat_cols], drop_first=True).astype(float)
train_means = X_en.mean()
X_en = X_en.fillna(train_means)

# Test set: same columns as train, missing values filled with train means
X_test_en = pd.get_dummies(test[feat_cols], drop_first=True).astype(float)
X_test_en = X_test_en.reindex(columns=X_en.columns, fill_value=0).fillna(train_m

# Model: scale, then Elastic Net (alpha and l1_ratio chosen by internal CV)
enet = make_pipeline(
    StandardScaler(),
    ElasticNetCV(
        l1_ratio=[0.1, 0.3, 0.5, 0.7, 0.9, 1],
        alphas=np.logspace(-4, 1, 100),
        cv=5,
        max_iter=20000,
        random_state=42,
    ),
)

# Cross-validated MSE (10-fold, repeated 5 times)
cv = RepeatedKFold(n_splits=10, n_repeats=5, random_state=42)
enet_scores = -cross_val_score(
    enet, X_en, y_en, cv=cv, scoring="neg_mean_squared_error", n_jobs=-1
)

enet_tuned_mse = enet_scores.mean()
enet_tuned_rmse = np.sqrt(enet_tuned_mse)
std_enet_tuned_mse = enet_scores.std()

print(
    f"Elastic Net\n MSE:{enet_tuned_mse:.4f}\n RMSE:{enet_tuned_rmse:.4f}\n S
)

```

```

# Fit on all training data and inspect the chosen settings and coefficients
enet.fit(X_en, y_en)
model = enet[-1]
print(f"Best alpha: {model.alpha_:.4f} | Best l1_ratio: {model.l1_ratio}")

coefs = pd.Series(model.coef_, index=X_en.columns)
kept = coefs[coefs != 0].sort_values(key=abs, ascending=False)
print(f"Kept {len(kept)} of {len(coefs)} features")
print(kept)

# Test-set predictions (kept in memory, not saved to a file)
enet_test_pred = enet.predict(X_test_en)

```

```

Elastic Net
MSE:0.8056
RMSE:0.8976
STD:0.202
Best alpha: 0.1205 | Best l1_ratio: 0.3
Kept 22 of 39 features
failures          -0.272856
goout             -0.107930
sex_M              0.103530
schoolsup_yes     -0.100975
Medu              0.064680
romantic_yes      -0.053590
Mjob_health        0.050625
studytime          0.050466
age               -0.044871
Mjob_other         -0.044554
health            -0.039064
Fjob_teacher       0.033697
famsup_yes        -0.028234
famsize_LE3        0.020643
higher_yes         0.020153
Mjob_services      0.014538
Fedu              0.013781
address_U          0.007952
Pstatus_T         -0.007846
reason_reputation  0.007280
reason_other       0.006253
Fjob_other         -0.004021
dtype: float64

```

Chosen model

```

In [31]: # Display the MSE comparisons as a table
results = pd.DataFrame(
    {
        "Baseline CV MSE": [
            xgb_base_mse,
            rf_base_mse,
            knn_base_mse,
            np.nan,
        ],
        "Tuned CV MSE": [
            xgb_tuned_mse,
            rf_tuned_mse,
            knn_tuned_mse,

```

```

enet_tuned_mse,
    ],
},
index=["XGBoost", "Random Forest", "KNN", "Elastic Net"],
)

print(results.to_string())

```

	Baseline CV MSE	Tuned CV MSE
XGBoost	0.939000	0.739816
Random Forest	0.770925	0.748045
KNN	1.020179	0.860931
Elastic Net	NaN	0.805614

```

In [34]: # Compare baseline and tuned cross-validation MSE
fig, ax = plt.subplots(figsize=(8, 5))

models = results.index
x = np.arange(len(models))
width = 0.35

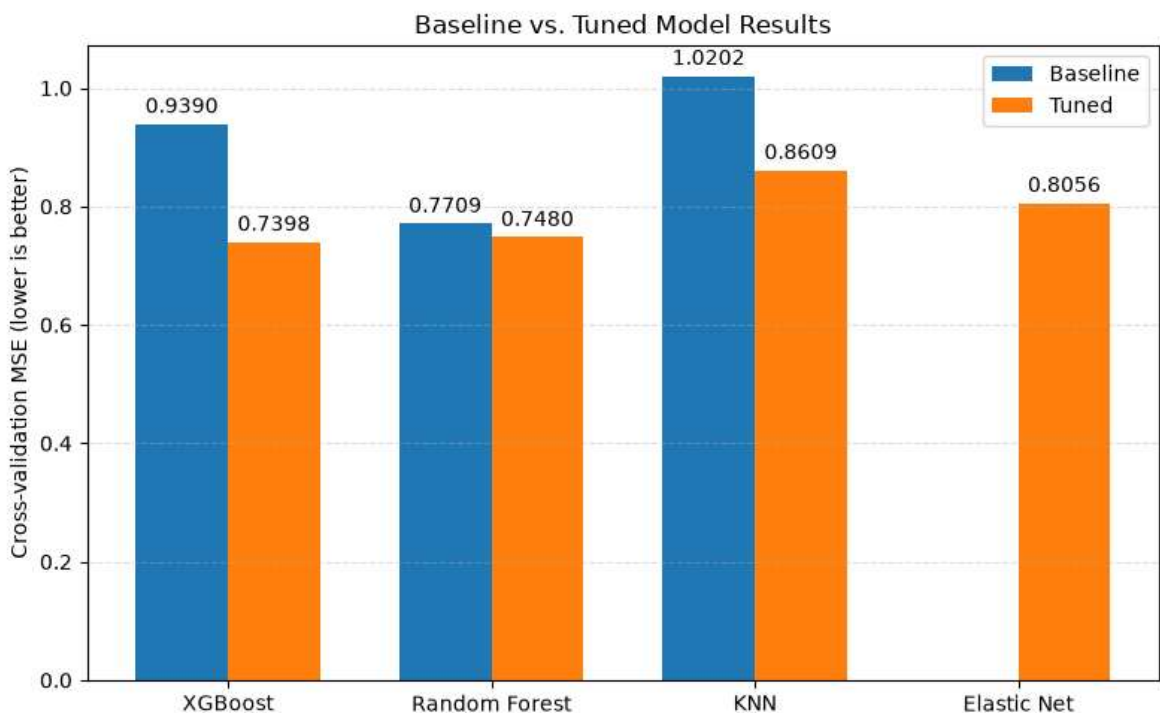
baseline = results["Baseline CV MSE"]
tuned = results["Tuned CV MSE"]

bars1 = ax.bar(x - width / 2, baseline, width, label="Baseline")
bars2 = ax.bar(x + width / 2, tuned, width, label="Tuned")

ax.set_xticks(x)
ax.set_xticklabels(models)
ax.set_ylabel("Cross-validation MSE (lower is better)")
ax.set_title("Baseline vs. Tuned Model Results")
ax.legend()
ax.bar_label(bars1, fmt="%.4f", padding=3)
ax.bar_label(bars2, fmt="%.4f", padding=3)
ax.grid(axis="y", linestyle="--", alpha=0.4)

plt.tight_layout()
plt.show()

```



Based on the cross-validation metrics, the tuned XGBoost is the best model because it achieves the lowest MSE (0.73). Lower MSE indicates smaller prediction errors thus higher accuracy. Random Forest provided the best base model in comparison to the others (0.77). In general hyperparameter tuning has provided us with better results especially for the XGBoost model, which surpassed Random Forest after some iterations.

Perform predictions

```
In [33]: X_test = test_df.copy()

# Create a new feature for the mean of parents' education
X_test["parent_edu_mean"] = (X_test["Medu"] + X_test["Fedu"]) / 2.0

# Convert nominal features to dummy variables
X_test = pd.get_dummies(
    X_test, columns=["Mjob", "Fjob", "reason", "guardian"], drop_first=True
)

# Fit the tuned XGBoost model on all the training data
xgb_tuned_model.fit(X_XGB, y)

# Predict on the test set
y_test_pred = xgb_tuned_model.predict(X_test)

# Save the test predictions as vector to an RDS file
r_vector = robjects.FloatVector(y_test_pred)
robjects.r["saveRDS"](r_vector, "predictions.rds")
```

```
Out[33]: <rp2.rinterface_lib.sexp.NULLType object at 0x0000018315525390> [0]
```

Team member contributions

We did most of the report together, the only part we split up was the models.

- Ossama Sijbesma (9890238): XGBoost, Model Comparison, Chosen Model
- Nóra Zala (): Random Forest, Model Description
- Rayana Sajith (): KNN, Model Description
- Divyadarsini Balasubramanian (): Elastic Net, Model Description